Lập trình mạng · Chương 1

Định danh: địa chỉ IP, tên miền, port và endpoint

TS. Đặng Ngọc Hùng

Chương 1: Tổng quan lập trình mạng

1 Định danh

Địa chỉ IP, tên miền, port và endpoint

2 Giao vận

TCP, UDP và mô hình ứng dụng mạng

3 Stream

Byte stream, contract của read/write, decorator và buffer

Dữ liệu có kiểu với DataStream và ObjectStream

Character stream, charset và framing message

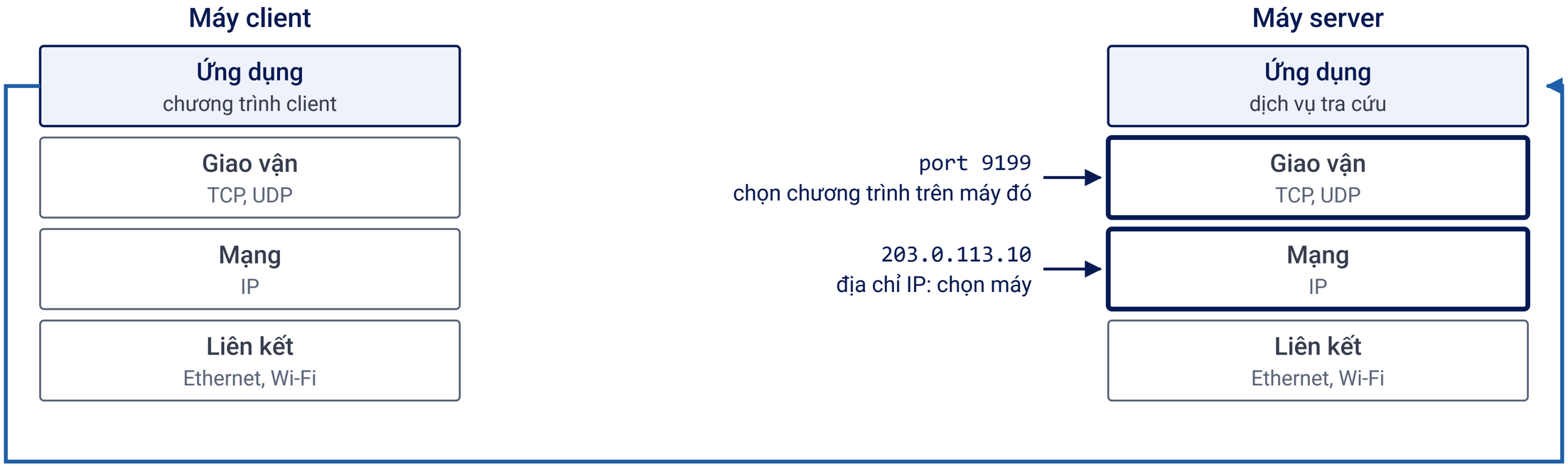
Bài toán ứng dụng: đặc tả và thử giao thức tra cứu

Muốn gửi dữ liệu, chương trình phải chỉ định được đích đến

1. Chương trình mạng gửi dữ liệu cho **một chương trình khác**, thường chạy trên một máy khác
2. Ví dụ xuyên suốt học phần: client gửi một dòng chứa khóa tới **dịch vụ tra cứu**, server trả lại một dòng kết quả

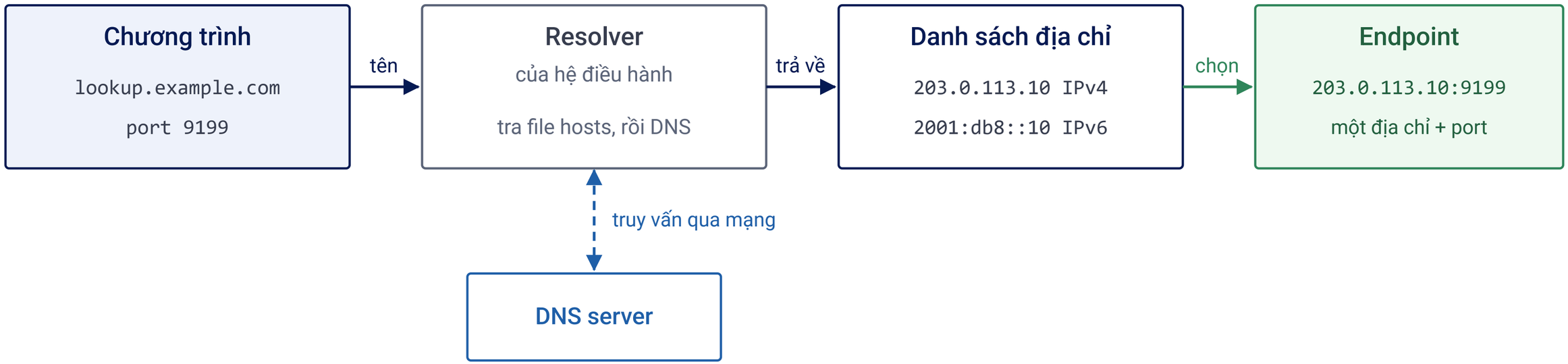
> **Câu hỏi của phần:** đích đến được chỉ định bằng những giá trị nào, và Java biểu diễn các giá trị đó ra sao?

Địa chỉ IP chọn máy, port chọn chương trình trên máy đó



Endpoint = địa chỉ IP + port: đích đến của dữ liệu là một chương trình trên một máy.

Phân giải tên đổi tên miền thành danh sách địa chỉ IP



Phân giải tên: tên miền → danh sách địa chỉ IP (IPv4, IPv6).

Endpoint = **một** địa chỉ trong danh sách + port.

Phương thức getAllByName trả về mọi địa chỉ của một tên

```java
InetAddress[] addresses = InetAddress.getAllByName("localhost");
for (InetAddress address : addresses) {
    System.out.println("localhost -> " + address.getHostAddress());
}
```

Kết quả chạy

```
localhost -> 127.0.0.1
localhost -> 0:0:0:0:0:0:0:1
```

- Một tên, hai địa chỉ: `127.0.0.1` (IPv4) và `::1` (IPv6, Java in đầy đủ `0:0:0:0:0:0:0:1`)

InetSocketAddress ghép địa chỉ với port thành endpoint

```java
var server = new InetSocketAddress("localhost", 9199);
System.out.println(server + "  isUnresolved() = " + server.isUnresolved());

var unknown = new InetSocketAddress("lookup.invalid", 9199);
System.out.println(unknown + "  isUnresolved() = " + unknown.isUnresolved());
```

Kết quả chạy

```
localhost/127.0.0.1:9199  isUnresolved() = false
lookup.invalid/<unresolved>:9199  isUnresolved() = true
```

- Tên không phân giải được **vẫn tạo được** đối tượng; `isUnresolved()` trả `true`

Phân giải tên thành công chưa cho biết có chương trình chờ ở port đó

1. `localhost/127.0.0.1:9199`, `isUnresolved() = false`: tên **đã có địa chỉ**
2. Lúc chạy, **không có chương trình nào** chờ ở port 9199 — chỉ biết được khi thiết lập kết nối ở tầng giao vận

Câu hỏi 1

Phân giải tên lookup.example.com thành công, được địa chỉ 203.0.113.10. Điều nào chắc chắn đúng?

A. Đang có chương trình chờ ở port 9199 trên máy đó.

B. Kết nối TCP tới máy đó đã được thiết lập.

C. Tên miền này chỉ có đúng một địa chỉ.

D. Đã có địa chỉ để tạo endpoint, nhưng chưa biết có chương trình nào chờ ở port 9199.

Câu hỏi 1

Phân giải tên lookup.example.com thành công, được địa chỉ 203.0.113.10. Điều nào chắc chắn đúng?

A. Đang có chương trình chờ ở port 9199 trên máy đó.

B. Kết nối TCP tới máy đó đã được thiết lập.

C. Tên miền này chỉ có đúng một địa chỉ.

D. Đã có địa chỉ để tạo endpoint, nhưng chưa biết có chương trình nào chờ ở port 9199.

Phân giải tên chỉ đổi tên thành địa chỉ, không liên lạc với chương trình ở đích. Chỉ khi thiết lập kết nối mới biết có chương trình chờ ở port hay không. Một tên có thể có nhiều địa chỉ.

Câu hỏi 2

Phương thức getAllByName trả về hai địa chỉ cho tên của server: một địa chỉ IPv4, một địa chỉ IPv6. Client nên làm gì?

A. Báo lỗi, vì một tên miền chỉ được có một địa chỉ.

B. Chọn một địa chỉ để kết nối; nếu không kết nối được thì thử địa chỉ tiếp theo.

C. Gửi dữ liệu tới cả hai địa chỉ cùng lúc để chắc chắn đến nơi.

D. Chỉ dùng địa chỉ IPv4, vì Java không kết nối được tới địa chỉ IPv6.

Câu hỏi 2

Phương thức getAllByName trả về hai địa chỉ cho tên của server: một địa chỉ IPv4, một địa chỉ IPv6. Client nên làm gì?

A. Báo lỗi, vì một tên miền chỉ được có một địa chỉ.

B. Chọn một địa chỉ để kết nối; nếu không kết nối được thì thử địa chỉ tiếp theo.

C. Gửi dữ liệu tới cả hai địa chỉ cùng lúc để chắc chắn đến nơi.

D. Chỉ dùng địa chỉ IPv4, vì Java không kết nối được tới địa chỉ IPv6.

Một tên có thể có nhiều địa chỉ (IPv4, IPv6, nhiều server). Mỗi endpoint chỉ gồm một địa chỉ và một port, nên client chọn một địa chỉ, không kết nối được thì thử địa chỉ tiếp theo. Java hỗ trợ cả IPv4 và IPv6.

Tổng kết

1. Đích đến là **endpoint** = địa chỉ IP (chọn máy) + port (chọn chương trình)

2. **Phân giải tên** do resolver của hệ điều hành thực hiện: tên miền → danh sách địa chỉ IPv4, IPv6

3. Java: `InetAddress` là địa chỉ, `InetSocketAddress` là endpoint; phân giải được chưa có nghĩa có chương trình chờ ở port

Từ khóa cần nhớ phân lớp giao thức · địa chỉ IP · tên miền · phân giải tên · DNS · port · endpoint · InetSocketAddress

Mở rộng

Chủ đề nâng cao

- IPv6 và dual-stack: thứ tự ưu tiên địa chỉ khi kết nối
- Cache DNS của JVM (networkaddress.cache.ttl)

Bài tập mở rộng

Viết chương trình nhận tên miền từ dòng lệnh, in mọi địa chỉ phân giải được và loại của từng địa chỉ (IPv4 hay IPv6, loopback hay không).

Tiếp theo

Giao vận: TCP, UDP và mô hình ứng dụng mạng